

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
TPHCM

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

ĐỀ TÀI:

HỆ THỐNG GIAO HÀNG BẰNG
DRONE THÔNG MINH

Giảng viên hướng dẫn: Nguyễn Văn Chiến

Nhóm sinh viên thực hiện: Nhóm 1

TP. HỒ CHÍ MINH - 2026

Mục lục

1	TỔNG QUAN VỀ ĐỀ TÀI	3
1.1	Lý do chọn đề tài	3
1.2	Khảo sát hiện trạng	3
1.3	Mục tiêu nghiên cứu	4
1.4	Phạm vi của đề án	4
1.5	Phương pháp thực hiện	4
2	PHÂN TÍCH YÊU CẦU HỆ THỐNG	6
2.1	Phân tích yêu cầu hệ thống	6
2.1.1	Yêu cầu chức năng	6
2.1.2	Yêu cầu phi chức năng	7
2.2	Sơ đồ Use Case	7
2.3	Mô hình hóa quy trình nghiệp vụ (BPMN)	7
2.3.1	Luồng 1: Quản lý yêu cầu giao hàng	7
2.3.2	Luồng 2: Quản lý kiện hàng	7
2.3.3	Luồng 3: Thực hiện giao hàng bằng drone	8
2.3.4	Luồng 4: Theo dõi trạng thái giao hàng	8
2.3.5	Luồng 5: Xác nhận hoàn thành giao hàng	8
2.4	Thiết kế giao diện người dùng (UI)	8
2.5	Thiết kế trải nghiệm người dùng (UX)	8
3	THIẾT KẾ KIẾN TRÚC PHẦN MỀM VÀ CƠ SỞ DỮ LIỆU	9
3.1	Giới thiệu kiến trúc hệ thống	9
3.1.1	Mô hình kiến trúc Clean Architecture	9
3.2	Thiết kế cơ sở dữ liệu PostgreSQL	10
3.2.1	Chi tiết các bảng dữ liệu	10
3.3	Mô tả luồng vận hành nghiệp vụ	12
3.4	Thiết kế giao diện lập trình ứng dụng RESTful API	12
3.4.1	Danh sách các cổng kết nối chính	13
3.4.2	Đặc tả định dạng dữ liệu truyền nhận	13

4	THIẾT KẾ GIAO DIỆN VÀ TRIỂN KHAI ỨNG DỤNG	15
4.1	Giới thiệu chương	15
4.2	Thiết kế giao diện UI/UX	15
4.2.1	Mục tiêu thiết kế	15
4.2.2	Design system	15
4.2.3	Thiết kế màn hình Mobile App	16
4.2.4	Thiết kế màn hình Web Portal	16
4.3	Triển khai Web Portal bằng ReactJS	17
4.3.1	Kiến trúc thư mục	17
4.3.2	Các thành phần chính	17
4.3.3	Quản lý state và gọi API	18
4.3.4	Phân quyền người dùng	18
4.4	Triển khai Mobile App bằng Flutter	18
4.4.1	Kiến trúc ứng dụng	18
4.4.2	Quản lý state	19
4.4.3	Tích hợp bản đồ	19
4.4.4	Push Notification	19
4.5	Tích hợp Real-time bằng SignalR	19
4.5.1	Lý do lựa chọn SignalR	19
4.5.2	Kiến trúc Hub	19
4.5.3	Luồng dữ liệu real-time	20
4.5.4	Xử lý mất kết nối	20
4.6	Kết chương	20
5	Triển Khai Dịch Vụ AI, QA/DevOps Và Đánh Giá Hệ Thống	21
5.1	Triển Khai Các Dịch Vụ AI	21
5.1.1	Dịch vụ Chatbot hỗ trợ khách hàng (Customer Support Chatbot)	21
5.1.2	Dịch vụ Dự đoán Thời gian Giao hàng (ETA Estimation)	22
5.1.3	Dịch vụ Tóm tắt Báo cáo Vận hành (Summarization)	22
5.2	Đóng Gói Hệ Thống Và Hạ Tầng DevOps	22
5.2.1	Cấu trúc Đóng gói Docker Container	22
5.2.2	Định hình tệp Dockerfile và Docker Compose	22
5.3	Kiểm Thử Phần Mềm (Software Testing)	23
5.3.1	Kế hoạch và Phương pháp Kiểm thử	24
5.3.2	Bảng Kết quả Kiểm thử Tổng hợp	24
5.4	Kết Luận Và Hướng Phát Triển	24
5.4.1	Kết luận	24
5.4.2	Hướng phát triển	24

Chương 1

TỔNG QUAN VỀ ĐỀ TÀI

1.1 Lý do chọn đề tài

Trong những năm gần đây, sự bùng nổ của thương mại điện tử đã kéo theo nhu cầu vận chuyển hàng hóa tăng cao đột biến. Tuy nhiên, mô hình giao hàng truyền thống bằng xe máy tại các đô thị lớn đang đối mặt với nhiều thách thức nghiêm trọng như tình trạng ùn tắc giao thông kéo dài, chi phí vận hành gia tăng và mức độ ô nhiễm môi trường ngày càng vượt ngưỡng cho phép. Đặc biệt, đối với các đơn hàng cần thời gian vận chuyển nhanh như vật tư y tế khẩn cấp, tài liệu quan trọng hay thực phẩm tươi sống, phương thức giao hàng đường bộ thường khó đảm bảo được mốc thời gian cam kết.

Sự phát triển của công nghệ tự động hóa và thiết bị bay không người lái mở ra một hướng tiếp cận hoàn toàn mới cho bài toán giao hàng chặng cuối. Khả năng di chuyển linh hoạt trên không giúp thiết bị dễ dàng vượt qua các điểm nghẽn giao thông mặt đất, rút ngắn tối đa thời gian vận chuyển và giảm thiểu chi phí nhân công. Xuất phát từ thực tiễn đó, nhóm nghiên cứu lựa chọn thực hiện đề tài xây dựng hệ thống quản lý và điều hành giao hàng bằng thiết bị bay thông minh nhằm giải quyết triệt để các hạn chế của mô hình vận chuyển hiện hành.

1.2 Khảo sát hiện trạng

Thực tế khảo sát tại thị trường Việt Nam cho thấy, các đơn vị vận tải hiện nay vẫn phụ thuộc gần như hoàn toàn vào đội ngũ tài xế công nghệ. Phương thức này bộc lộ rõ nhược điểm khi phụ thuộc lớn vào mật độ giao thông theo từng khung giờ trong ngày, dẫn đến năng suất lao động giảm mạnh vào các giờ cao điểm. Đồng thời, việc biến động giá nhiên liệu và chi phí nhân công cũng tạo ra áp lực tài chính không nhỏ lên các doanh nghiệp vận tải.

Trên thế giới, những tập đoàn công nghệ lớn đã triển khai thử nghiệm thành công

mô hình giao hàng bằng thiết bị bay tại một số khu vực. Mặc dù vậy, tại thị trường trong nước, các giải pháp phần mềm mang tính hệ thống nhằm điều hành, giám sát hạm đội thiết bị tập trung và kết nối đa nền tảng vẫn chưa được nghiên cứu toàn diện. Việc xây dựng một giải pháp phần mềm làm chủ quy trình vận hành này là đòi hỏi cấp thiết.

1.3 Mục tiêu nghiên cứu

Đề tài hướng đến mục tiêu thiết kế và triển khai một hệ thống phần mềm toàn diện nhằm quản lý khép kín quy trình giao hàng. Về mặt hạ tầng máy chủ, hệ thống xây dựng nền tảng dịch vụ backend dựa trên framework ASP.NET Core kết hợp cơ sở dữ liệu PostgreSQL để lưu trữ, xử lý thông tin người dùng, đơn hàng và quản lý trạng thái của danh sách thiết bị bay.

Ở góc độ điều hành và tương tác người dùng, dự án phát triển cổng thông tin quản trị bằng ReactJS, tích hợp bản đồ số để hỗ trợ nhân viên giám sát tọa độ di chuyển của hạm đội thiết bị theo thời gian thực. Đồng thời, ứng dụng di động xây dựng trên nền tảng Flutter cho phép người dùng cuối chủ động đặt đơn, theo dõi lộ trình bay và nhận thông báo khi hàng tới điểm đích. Ngoài ra, hệ thống cũng tích hợp thuật toán ước tính thời gian giao hàng cùng công cụ hỗ trợ giải đáp thắc mắc tự động cho khách hàng.

1.4 Phạm vi của đề án

Về mặt nghiệp vụ, dự án tập trung xử lý luồng giao hàng chặng cuối đối với các kiện hàng có khối lượng nhỏ và vừa, hoạt động trong bán kính thử nghiệm cho phép của trạm điều hành.

Về mặt kỹ thuật, hệ thống sử dụng framework ASP.NET Core phiên bản mới nhất cho phần máy chủ, kết hợp công nghệ SignalR để truyền nhận dữ liệu vị trí theo thời gian thực. Trang quản trị sử dụng ReactJS kết hợp thư viện Leaflet cho việc hiển thị bản đồ. Ứng dụng di động được hoàn thiện bằng Flutter nhằm hỗ trợ tốt trên hệ điều hành Android. Toàn bộ giải pháp được đóng gói và triển khai trên môi trường Docker để đảm bảo tính đóng gói và khả năng mở rộng trong tương lai.

1.5 Phương pháp thực hiện

Quá trình nghiên cứu và phát triển đề tài được tiến hành qua nhiều giai đoạn có tính kết nối chặt chẽ. Ban đầu, nhóm tiến hành khảo sát yêu cầu thực tế, xây dựng mô hình dữ liệu tổng thể và thiết kế chi tiết luồng nghiệp vụ của từng thành phần trong hệ thống.

Để quản lý dự án hiệu quả, nhóm áp dụng công cụ Git và GitHub nhằm đồng bộ mã nguồn giữa các thành viên, tránh tình trạng xung đột trong quá trình phát triển. Cấu trúc thư mục của dự án được phân chia rõ ràng với thư mục lưu trữ riêng cho tài liệu báo cáo và thư mục chứa mã nguồn ứng dụng, giúp việc lập trình và hoàn thiện báo cáo được thực hiện song song một cách khoa học.

Chương 2

PHÂN TÍCH YÊU CẦU HỆ THỐNG

2.1 Phân tích yêu cầu hệ thống

2.1.1 Yêu cầu chức năng

Hệ thống Smart Drone Delivery Management Platform cần đáp ứng các yêu cầu chức năng sau.

Quản lý thông tin khách hàng: Hệ thống cho phép khách hàng đăng ký tài khoản, đăng nhập và cập nhật thông tin cá nhân nhằm phục vụ quá trình sử dụng dịch vụ.

Quản lý yêu cầu giao hàng: Khách hàng có thể tạo mới, chỉnh sửa và theo dõi yêu cầu giao hàng với đầy đủ thông tin về người gửi, người nhận và địa điểm giao nhận.

Quản lý kiện hàng: Hệ thống lưu trữ các thông tin của kiện hàng như khối lượng, kích thước, loại hàng hóa và địa điểm nhận, địa điểm giao.

Phân công drone: Hệ thống hỗ trợ phân công drone phù hợp để thực hiện giao hàng dựa trên trạng thái hoạt động và khả năng vận chuyển của từng drone.

Theo dõi giao hàng: Khách hàng có thể theo dõi trạng thái đơn hàng và vị trí của drone theo thời gian thực.

Xác nhận giao hàng: Sau khi hoàn tất quá trình giao hàng, khách hàng xác nhận đã nhận hàng và hệ thống cập nhật trạng thái đơn hàng.

Quản lý trạm drone: Hệ thống quản lý thông tin các trạm cất cánh và hạ cánh của drone nhằm hỗ trợ quá trình điều phối.

Quản lý đội drone: Theo dõi tình trạng hoạt động, bảo trì và khả năng vận chuyển của từng drone.

Thống kê và báo cáo: Cung cấp các báo cáo về số lượng đơn hàng, doanh thu, hiệu suất hoạt động và tình trạng sử dụng drone.

Quản lý người dùng: Quản trị viên có thể quản lý tài khoản và phân quyền cho từng nhóm người dùng.

2.1.2 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng các yêu cầu phi chức năng sau.

Hiệu năng: Hệ thống phải có thời gian phản hồi nhanh, đảm bảo xử lý các thao tác thông thường trong thời gian ngắn.

Bảo mật: Thông tin người dùng được bảo vệ thông qua cơ chế xác thực và phân quyền truy cập.

Độ tin cậy: Hệ thống hoạt động ổn định, hạn chế lỗi và đảm bảo tính sẵn sàng trong quá trình vận hành.

Khả năng mở rộng: Hệ thống có thể mở rộng khi số lượng người dùng, drone và đơn hàng tăng lên.

Khả năng bảo trì: Dễ dàng cập nhật, nâng cấp và sửa lỗi mà không ảnh hưởng đến toàn bộ hệ thống.

An toàn dữ liệu: Dữ liệu được sao lưu định kỳ nhằm hạn chế rủi ro mất mát thông tin.

2.2 Sơ đồ Use Case

Hệ thống gồm năm tác nhân chính là Khách hàng (Customer), Nhân viên điều phối (Dispatcher), Nhân viên trạm (Station Operator), Quản lý Logistics và Quản trị viên (Administrator).

Các chức năng chính của hệ thống bao gồm đăng ký tài khoản, đăng nhập, tạo yêu cầu giao hàng, quản lý kiện hàng, phân công drone, theo dõi trạng thái giao hàng, xác nhận giao hàng, quản lý drone, quản lý trạm, thống kê báo cáo và quản lý người dùng.

2.3 Mô hình hóa quy trình nghiệp vụ (BPMN)

2.3.1 Luồng 1: Quản lý yêu cầu giao hàng

Khách hàng đăng nhập vào hệ thống và nhập thông tin đơn hàng. Hệ thống kiểm tra tính hợp lệ của dữ liệu, sau đó tạo yêu cầu giao hàng và chuyển đến nhân viên điều phối để xử lý.

2.3.2 Luồng 2: Quản lý kiện hàng

Nhân viên tiếp nhận kiện hàng, kiểm tra khối lượng và kích thước, cập nhật thông tin vào hệ thống và chờ phân công drone thực hiện giao hàng.

2.3.3 Luồng 3: Thực hiện giao hàng bằng drone

Nhân viên điều phối lựa chọn drone phù hợp và gửi lệnh thực hiện giao hàng. Drone cất cánh, di chuyển đến địa điểm giao và hoàn thành việc giao kiện hàng cho khách hàng.

2.3.4 Luồng 4: Theo dõi trạng thái giao hàng

Trong quá trình vận chuyển, drone liên tục gửi dữ liệu vị trí GPS về hệ thống. Hệ thống cập nhật trạng thái giao hàng để khách hàng có thể theo dõi theo thời gian thực.

2.3.5 Luồng 5: Xác nhận hoàn thành giao hàng

Sau khi giao hàng thành công, khách hàng xác nhận đã nhận hàng. Hệ thống cập nhật trạng thái đơn hàng là “Hoàn thành” và lưu trữ lịch sử giao hàng.

2.4 Thiết kế giao diện người dùng (UI)

Hệ thống được thiết kế với các giao diện chính gồm màn hình đăng nhập, màn hình đăng ký, trang chủ, giao diện tạo yêu cầu giao hàng, danh sách đơn hàng, theo dõi vị trí drone, quản lý drone, quản lý trạm, thống kê báo cáo và giao diện quản trị hệ thống. Các giao diện được thiết kế đơn giản, trực quan và dễ sử dụng.

2.5 Thiết kế trải nghiệm người dùng (UX)

Trải nghiệm người dùng được xây dựng theo hướng đơn giản và thuận tiện. Quy trình tạo đơn hàng được rút gọn nhằm giảm số bước thao tác. Hệ thống hiển thị trạng thái giao hàng theo thời gian thực, đảm bảo giao diện thống nhất trên các màn hình và hỗ trợ sử dụng trên cả máy tính lẫn thiết bị di động.

Chương 3

THIẾT KẾ KIẾN TRÚC PHẦN MỀM VÀ CƠ SỞ DỮ LIỆU

3.1 Giới thiệu kiến trúc hệ thống

Nền tảng Quản lý Giao hàng bằng Drone Thông minh được thiết kế nhằm mục tiêu tự động hóa toàn bộ quy trình vận hành giao nhận hàng hóa. Hệ thống chịu trách nhiệm điều phối các thiết bị bay không người lái, tối ưu hóa hoạt động tại các trạm đáp và nâng cao độ chính xác trong công tác quản lý vận chuyển.

3.1.1 Mô hình kiến trúc Clean Architecture

Hệ thống phía máy chủ (Backend) được xây dựng trên nền tảng ASP.NET Core Web API và áp dụng mô hình Clean Architecture. Việc phân chia hệ thống thành các tầng độc lập giúp đảm bảo tính mô-đun, dễ dàng mở rộng và thuận tiện trong quá trình kiểm thử phần mềm.

1. Tầng Domain (Domain Layer): Tầng cốt lõi chứa các đối tượng nghiệp vụ chính của hệ thống như người dùng, đơn hàng, thiết bị drone và trạm đáp. Các quy tắc nghiệp vụ cơ bản được định nghĩa hoàn toàn độc lập tại đây mà không phụ thuộc vào bất kỳ thư viện hay công nghệ lưu trữ dữ liệu nào bên ngoài.

2. Tầng Application (Application Layer): Tầng ứng dụng chịu trách nhiệm xử lý các luồng công việc cụ thể bao gồm tạo đơn hàng, phê duyệt đơn, điều phối thiết bị và tính toán thời gian giao hàng dự kiến. Hệ thống áp dụng mẫu thiết kế CQRS kết hợp thư viện MediatR để quản lý và xử lý các lệnh cũng như truy vấn dữ liệu một cách hiệu quả.

3. Tầng Infrastructure (Tầng hạ tầng): Tầng hạ tầng đảm nhận việc triển khai thực tế các giao diện được khai báo ở tầng ứng dụng. Dữ liệu hệ thống được quản lý thông qua hệ quản trị cơ sở dữ liệu PostgreSQL với Entity Framework Core. Các tệp tin hình ảnh được lưu trữ trên hệ thống MinIO Object Storage, trong khi các thông

tin theo dõi thời gian thực được truyền tải qua giao thức SignalR.

4. Tầng Presentation (Tầng hiển thị): Lớp giao diện cung cấp các cổng kết nối RESTful API mã hóa dữ liệu theo chuẩn JSON. Cổng API này phục vụ trực tiếp cho ứng dụng quản trị trên nền tảng Web xây dựng bằng ReactJS và ứng dụng di động dành cho người dùng phát triển trên Flutter.

3.2 Thiết kế cơ sở dữ liệu PostgreSQL

Cơ sở dữ liệu của hệ thống được chuẩn hóa theo dạng chuẩn 3NF trên hệ quản trị PostgreSQL nhằm đảm bảo tính toàn vẹn dữ liệu và tối ưu hiệu năng truy vấn. Các bảng dữ liệu chính trong hệ thống được chi tiết hóa như sau.

3.2.1 Chi tiết các bảng dữ liệu

1. Bảng Users (Người dùng): Lưu trữ thông tin tài khoản, thông tin liên lạc và quyền hạn của người dùng trong hệ thống.

Thuộc tính	Kiểu dữ liệu	Mô tả / Ràng buộc
user_id	UUID	Khóa chính
full_name	VARCHAR(100)	Họ và tên người dùng
email	VARCHAR(100)	Địa chỉ email (Duy nhất)
phone_number	VARCHAR(20)	Số điện thoại
password_hash	TEXT	Mật khẩu đã mã hóa
role	VARCHAR(30)	Vai trò người dùng
created_at	TIMESTAMP	Thời gian tạo tài khoản

Bảng 3.1: Cấu trúc bảng Users

2. Bảng Packages (Kiện hàng): Quản lý các thông số vật lý của kiện hàng cần vận chuyển bao gồm kích thước, trọng lượng và các đặc tính bảo quản.

Thuộc tính	Kiểu dữ liệu	Mô tả / Ràng buộc
package_id	UUID	Khóa chính
weight_kg	DECIMAL(5,2)	Trọng lượng kiện hàng (kg)
length_cm	DECIMAL(5,2)	Chiều dài (cm)
width_cm	DECIMAL(5,2)	Chiều rộng (cm)
height_cm	DECIMAL(5,2)	Chiều cao (cm)
is_fragile	BOOLEAN	Cờ đánh dấu hàng dễ vỡ
image_url	TEXT	Đường dẫn ảnh kiện hàng

Bảng 3.2: Cấu trúc bảng Packages

3. Bảng Landing Stations (Trạm đáp): Quản lý vị trí tọa độ địa lý, sức chứa và trạng thái hoạt động của các trạm cất/hạ cánh.

Thuộc tính	Kiểu dữ liệu	Mô tả / Ràng buộc
station_id	UUID	Khóa chính
station_name	VARCHAR(100)	Tên trạm đáp
address_text	TEXT	Địa chỉ chi tiết
latitude	DECIMAL(10,7)	Vĩ độ GPS
longitude	DECIMAL(10,7)	Kinh độ GPS
max_capacity	INTEGER	Sức chứa Drone tối đa
status	VARCHAR(30)	Trạng thái hoạt động trạm

Bảng 3.3: Cấu trúc bảng Landing Stations

4. Bảng Drones (Thiết bị bay): Quản lý thông số kỹ thuật của từng drone, mức dung lượng pin hiện tại và tải trọng thiết kế.

Thuộc tính	Kiểu dữ liệu	Mô tả / Ràng buộc
drone_id	UUID	Khóa chính
model_code	VARCHAR(50)	Mã dòng thiết bị
max_payload_kg	DECIMAL(5,2)	Tải trọng tối đa (kg)
battery_capacity_pct	INTEGER	Dung lượng pin (%)
current_station_id	UUID	Khóa ngoại tham chiếu Trạm đáp
status	VARCHAR(30)	Trạng thái hoạt động Drone

Bảng 3.4: Cấu trúc bảng Drones

5. Bảng Delivery Orders (Đơn hàng giao): Lưu trữ thông tin chi tiết các đơn giao hàng do người dùng khởi tạo trên hệ thống.

Thuộc tính	Kiểu dữ liệu	Mô tả / Ràng buộc
order_id	UUID	Khóa chính
customer_id	UUID	Khóa ngoại tham chiếu Người dùng
package_id	UUID	Khóa ngoại tham chiếu Kịch bản hàng
pickup_station_id	UUID	Khóa ngoại tham chiếu Trạm lấy hàng
destination_address	TEXT	Địa chỉ điểm giao
dest_latitude	DECIMAL(10,7)	Vĩ độ điểm giao
dest_longitude	DECIMAL(10,7)	Kinh độ điểm giao
status	VARCHAR(30)	Trạng thái đơn hàng
created_at	TIMESTAMP	Thời gian khởi tạo

Bảng 3.5: Cấu trúc bảng Delivery Orders

6. Bảng Deliveries (Chuyến giao hàng): Ghi nhận quá trình phân công thiết bị bay thực hiện nhiệm vụ cho một đơn hàng cụ thể.

7. Bảng Tracking Logs (Nhật ký hành trình): Ghi lại lịch sử tọa độ GPS, độ cao và tốc độ di chuyển của drone theo thời gian thực.

Thuộc tính	Kiểu dữ liệu	Mô tả / Ràng buộc
delivery_id	UUID	Khóa chính
order_id	UUID	Khóa ngoại tham chiếu Đơn hàng
drone_id	UUID	Khóa ngoại tham chiếu Drone
dispatcher_id	UUID	Khóa ngoại tham chiếu Điều phối viên
estimated_eta_minutes	INTEGER	Thời gian giao dự kiến (phút)
confirmation_code	VARCHAR(10)	Mã xác nhận giao hàng OTP
start_time	TIMESTAMP	Thời điểm cất cánh
end_time	TIMESTAMP	Thời điểm hoàn thành

Bảng 3.6: Cấu trúc bảng Deliveries

Thuộc tính	Kiểu dữ liệu	Mô tả / Ràng buộc
log_id	BIGSERIAL	Khóa chính tự tăng
delivery_id	UUID	Khóa ngoại tham chiếu Chuyển giao
current_latitude	DECIMAL(10,7)	Vĩ độ hiện tại
current_longitude	DECIMAL(10,7)	Kinh độ hiện tại
current_altitude_m	DECIMAL(6,2)	Độ cao hiện tại (mét)
battery_remaining_pct	INTEGER	Mức pin còn lại (%)
speed_kmh	DECIMAL(5,2)	Tốc độ bay (km/h)
timestamp	TIMESTAMP	Thời điểm ghi nhật ký

Bảng 3.7: Cấu trúc bảng Tracking Logs

3.3 Mô tả luồng vận hành nghiệp vụ

Quy trình giao vận bằng drone được thực hiện qua các bước tuần tự. Ban đầu, khách hàng sử dụng ứng dụng di động để gửi yêu cầu đặt đơn kèm vị trí tọa độ điểm nhận và thông tin kiện hàng. Sau khi tiếp nhận, hệ thống gửi dữ liệu đến mô-đun tính toán để xác định khoảng cách, độ an toàn luồng bay và dự báo thời gian giao hàng.

Khi điều phối viên duyệt đơn hàng, hệ thống tự động quét và lựa chọn drone phù hợp tại trạm gần nhất dựa trên tiêu chí dung lượng pin và trọng tải cho phép. Sau khi nhận lệnh, drone bắt đầu hành trình và liên tục gửi thông số tọa độ GPS cùng trạng thái thiết bị về hệ thống máy chủ. Khi drone tới điểm giao, người dùng thực hiện xác thực mã OTP trên ứng dụng để nhận hàng. Hoàn tất quy trình, thiết bị quay trở lại trạm đáp ban đầu.

3.4 Thiết kế giao diện lập trình ứng dụng RESTful API

Các giao tiếp giữa ứng dụng client và máy chủ backend được thực hiện qua cổng API chuẩn RESTful, yêu cầu xác thực bằng chuỗi JWT Bearer Token.

3.4.1 Danh sách các cổng kết nối chính

Phương thức	Đường dẫn API	Chức năng nghiệp vụ
POST	/api/v1/auth/login	Đăng nhập và nhận Token
POST	/api/v1/orders	Đặt đơn giao hàng mới
GET	/api/v1/orders/{id}	Truy vấn thông tin đơn hàng
PATCH	/api/v1/orders/{id}/approve	Phê duyệt đơn hàng
GET	/api/v1/stations	Lấy danh sách các trạm đáp
GET	/api/v1/drones/available	Tìm kiếm drone đang khả dụng
POST	/api/v1/deliveries/dispatch	Phân công drone thực hiện chuyển
POST	/api/v1/deliveries/{id}/confirm	Xác nhận hoàn thành giao hàng

Bảng 3.8: Danh sách RESTful API Endpoints

3.4.2 Đặc tả định dạng dữ liệu truyền nhận

Dưới đây là ví dụ cấu trúc dữ liệu gửi lên máy chủ khi người dùng khởi tạo đơn hàng mới thông qua phương thức POST tới cổng /api/v1/orders:

```
{
  "pickup_station_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "destination_address": "Khu Công nghệ cao, Quận 9, TP.HCM",
  "dest_latitude": 10.854881,
  "dest_longitude": 106.785023,
  "package": {
    "weight_kg": 1.8,
    "length_cm": 20.0,
    "width_cm": 15.0,
    "height_cm": 10.0,
    "is_fragile": false
  }
}
```

Kết quả phản hồi từ máy chủ báo tạo đơn thành công trả về mã 201 Created có cấu trúc như sau:

```
{
  "success": true,
  "order_id": "c71a3d9b-8e21-4f1b-b72e-8301132a03e1",
  "status": "Pending",
  "created_at": "2026-09-08T01:30:00Z",
}
```

```
    "message": "Delivery order created successfully."
}
```

Dữ liệu cập nhật vị trí GPS trực tuyến của thiết bị gửi về hệ thống được định dạng như sau:

```
{
  "delivery_id": "f82b11a4-92c2-421d-9321-7290ad11bc82",
  "drone_id": "d1109a5f-4221-4b1a-8212-98442211aa09",
  "current_latitude": 10.852104,
  "current_longitude": 106.781200,
  "altitude_m": 45.5,
  "speed_kmh": 32.0,
  "battery_pct": 82
}
```

Chương 4

THIẾT KẾ GIAO DIỆN VÀ TRIỂN KHAI ỨNG DỤNG

4.1 Giới thiệu chương

Chương này trình bày quá trình thiết kế giao diện người dùng và triển khai các thành phần frontend của hệ thống Smart Drone Delivery, bao gồm Web Portal dành cho quản trị viên/điều phối viên và Mobile App dành cho khách hàng. Bên cạnh đó, chương cũng trình bày cách nhóm tích hợp SignalR để hiện thực hóa tính năng theo dõi drone theo thời gian thực, vốn là một trong những yêu cầu quan trọng nhất của hệ thống.

4.2 Thiết kế giao diện UI/UX

4.2.1 Mục tiêu thiết kế

Nhóm em thực hiện thiết kế giao diện trên Figma để thống nhất về màu sắc, bố cục và luồng thao tác giữa các thành viên, tránh tình trạng mỗi người code một kiểu giao diện khác nhau. Mục tiêu chính là giao diện phải đơn giản, dễ dùng vì đối tượng sử dụng app khách hàng có thể không rành công nghệ, còn Web Portal thì cần hiển thị được nhiều thông tin cùng lúc (bản đồ, danh sách đơn, trạng thái drone) mà vẫn rõ ràng, không rối mắt.

4.2.2 Design system

Nhóm xây dựng một bộ design system dùng chung cho cả Web và Mobile để đảm bảo tính đồng nhất:

- Màu chủ đạo: red (thể hiện sự tin cậy, công nghệ), kết hợp với các màu trạng thái như xanh lá (đã giao thành công), vàng cam (đang giao), đỏ (lỗi/hủy đơn)

- Font chữ: sử dụng font sans-serif (Inter/Roboto) cho dễ đọc trên nhiều kích thước màn hình
- Các component dùng chung: Button, Card, Badge trạng thái, Input field, Modal xác nhận

4.2.3 Thiết kế màn hình Mobile App

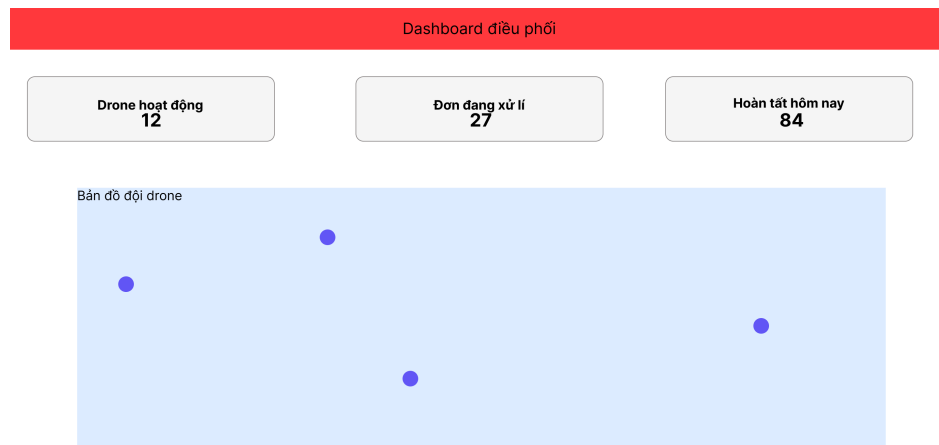
Mobile App phục vụ đối tượng khách hàng, các màn hình chính bao gồm:

- Màn hình Đăng nhập/Đăng ký
- Màn hình Tạo đơn giao hàng: khách chọn điểm nhận và điểm giao trên bản đồ
- Màn hình Theo dõi đơn hàng: hiển thị vị trí drone theo thời gian thực trên bản đồ
- Màn hình Lịch sử đơn hàng
- Màn hình Chatbot hỗ trợ khách hàng
- Màn hình Thông báo

4.2.4 Thiết kế màn hình Web Portal

Web Portal phục vụ đối tượng quản trị viên và điều phối viên, gồm các màn hình chính:

- Dashboard: tổng quan số lượng drone đang hoạt động, số đơn đang xử lý, số đơn hoàn thành trong ngày
- Bản đồ quản lý đội drone: hiển thị vị trí tất cả drone đang hoạt động trên cùng một bản đồ
- Quản lý đơn hàng: danh sách đơn, trạng thái, cho phép điều phối viên can thiệp khi cần
- Quản lý drone: thông tin pin, trạng thái hoạt động, lịch bảo trì
- Quản lý người dùng



Hình 4.1: Giao diện Dashboard của Web Portal

4.3 Triển khai Web Portal bằng ReactJS

4.3.1 Kiến trúc thư mục

Nhóm tổ chức source code của Web Portal theo cấu trúc sau để dễ quản lý khi nhiều người cùng code:

```
src/  
  components/  
  pages/  
  services/  
  hooks/  
  contexts/  
  utils/  
  assets/
```

Trong đó, thư mục **services** chứa các hàm gọi API xuống backend (đã thiết kế ở Chương 3), thư mục **components** chứa các thành phần giao diện dùng lại được như bảng, thẻ trạng thái, thư mục **pages** chứa các trang hoàn chỉnh tương ứng với từng route.

4.3.2 Các thành phần chính

Một số component quan trọng nhóm đã xây dựng:

- **DroneMap**: component hiển thị bản đồ, nhận dữ liệu vị trí drone theo thời gian thực và cập nhật marker trên bản đồ
- **OrderTable**: bảng hiển thị danh sách đơn hàng, hỗ trợ lọc theo trạng thái
- **DroneStatusCard**: thẻ hiển thị thông tin nhanh về một drone (pin, trạng thái, đơn đang giao)
- **Dashboard**: trang tổng quan, tổng hợp số liệu từ các API thống kê

4.3.3 Quản lý state và gọi API

Nhóm sử dụng Context API kết hợp với custom hooks để quản lý state dùng chung như thông tin người dùng đăng nhập, danh sách drone đang hoạt động, tránh phải truyền props qua nhiều tầng component. Việc gọi API xuống backend được thực hiện qua Axios, các request đều được đính kèm JWT token trong header **Authorization** để xác thực.

Ví dụ đoạn code gọi API lấy danh sách đơn hàng:

```
export const getOrders = async (status) => {
  const response = await axiosClient.get('/api/orders', {
    params: { status },
  });
  return response.data;
};
```

4.3.4 Phân quyền người dùng

Web Portal có hai vai trò chính là Quản trị viên (Admin) và Điều phối viên (Dispatcher), mỗi vai trò được giới hạn truy cập vào các trang khác nhau thông qua route bảo vệ (Protected Route) dựa trên role được giải mã từ JWT token.

4.4 Triển khai Mobile App bằng Flutter

4.4.1 Kiến trúc ứng dụng

Mobile App được xây dựng bằng Flutter, tổ chức theo hướng tách biệt các tầng để khớp với kiến trúc Clean Architecture đã trình bày ở Chương 3, gồm tầng presentation (giao diện), tầng domain (logic nghiệp vụ) và tầng data (gọi API, xử lý dữ liệu).

```
lib/
  presentation/
```

```
screens/  
widgets/  
domain/  
  entities/  
  usecases/  
data/  
  repositories/  
  datasources/
```

4.4.2 Quản lý state

Nhóm sử dụng thư viện quản lý state (Provider/BLoC) để tách biệt logic xử lý ra khỏi giao diện, giúp việc cập nhật vị trí drone theo thời gian thực không làm ảnh hưởng đến các phần khác của màn hình.

4.4.3 Tích hợp bản đồ

Màn hình theo dõi đơn hàng sử dụng Google Maps SDK để hiển thị bản đồ, vị trí drone được vẽ dưới dạng marker và cập nhật liên tục khi có dữ liệu mới gửi về từ server.

4.4.4 Push Notification

Ứng dụng tích hợp Firebase Cloud Messaging để gửi thông báo đẩy cho khách hàng khi trạng thái đơn hàng thay đổi, ví dụ như khi drone bắt đầu giao hàng hoặc đã giao hàng thành công.

4.5 Tích hợp Real-time bằng SignalR

4.5.1 Lý do lựa chọn SignalR

Ban đầu nhóm có cân nhắc dùng cách polling, tức là client tự động gọi API liên tục để hỏi vị trí drone mới nhất, nhưng cách này gây tốn tài nguyên server và độ trễ hiển thị không tốt. Vì vậy nhóm quyết định sử dụng SignalR để server có thể chủ động đẩy dữ liệu xuống client ngay khi có cập nhật, giúp việc theo dõi drone mượt mà và gần như tức thời hơn.

4.5.2 Kiến trúc Hub

Nhóm xây dựng một Hub tên là `DroneTrackingHub`, client sẽ subscribe vào một nhóm (group) tương ứng với mã đơn hàng (`orderId`) mà mình đang theo dõi. Khi drone gửi

vị trí GPS mới về server, backend sẽ xử lý và broadcast dữ liệu này đến đúng nhóm client đang theo dõi đơn hàng đó, thay vì gửi cho tất cả mọi người.

```
public class DroneTrackingHub : Hub
{
    public async Task JoinOrderGroup(string orderId)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId, orderId);
    }
}
```

4.5.3 Luồng dữ liệu real-time

Luồng cập nhật vị trí drone diễn ra như sau: drone định kỳ gửi tọa độ GPS về backend, backend nhận dữ liệu, xử lý và gọi đến `DroneTrackingHub` để broadcast xuống các client (cả Web Portal và Mobile App) đang lắng nghe nhóm đơn hàng tương ứng, sau đó client nhận dữ liệu và cập nhật vị trí marker trên bản đồ theo thời gian thực.

4.5.4 Xử lý mất kết nối

Vì drone hoạt động ngoài trời nên có khả năng gặp tình trạng mất sóng tạm thời, nhóm đã cấu hình cơ chế tự động kết nối lại (automatic reconnect) của SignalR ở phía client để đảm bảo khi mạng ổn định trở lại thì việc theo dõi vẫn được tiếp tục mà người dùng không cần tự tải lại trang hoặc mở lại ứng dụng.

4.6 Kết chương

Qua chương này, nhóm em đã trình bày quá trình thiết kế giao diện và triển khai hai nền tảng Web Portal, Mobile App, cùng với việc tích hợp SignalR để hiện thực hóa tính năng theo dõi drone theo thời gian thực. Đây là phần trực tiếp tương tác với người dùng cuối nên nhóm đã cố gắng cân bằng giữa tính thẩm mỹ và tính dễ sử dụng của giao diện.

Chương 5

Triển Khai Dịch Vụ AI, QA/DevOps Và Đánh Giá Hệ Thống

Chương này trình bày chi tiết việc thiết kế, huấn luyện và triển khai các dịch vụ Trí tuệ nhân tạo (AI) hỗ trợ hệ thống giao hàng bằng Drone thông minh, cùng hạ tầng DevOps, quy trình đóng gói phần mềm bằng Docker và công tác kiểm thử chất lượng (QA/Testing).

5.1 Triển Khai Các Dịch Vụ AI

Hệ thống giao hàng bằng Drone thông minh tích hợp ba dịch vụ AI cốt lõi nhằm tối ưu hóa trải nghiệm khách hàng và vận hành hạ tầng giao vận.

5.1.1 Dịch vụ Chatbot hỗ trợ khách hàng (Customer Support Chatbot)

Dịch vụ Chatbot đóng vai trò tiếp nhận phản hồi, giải đáp thắc mắc và hỗ trợ tra cứu trạng thái đơn hàng thời gian thực.

Mô hình triển khai: Sử dụng kiến trúc RAG (Retrieval-Augmented Generation) kết hợp với các mô hình ngôn ngữ lớn (LLM) thông qua API để đảm bảo câu trả lời bám sát dữ liệu vận hành nội bộ.

Tính năng chính: Trả lời thắc mắc về chính sách giao hàng, tải trọng tối đa của drone, và khu vực hỗ trợ. Truy vấn trạng thái chuyến bay của drone dựa trên Mã đơn hàng (Order ID).

Giao tiếp API: Xây dựng bằng FastAPI, trả về kết quả dạng JSON hỗ trợ tích hợp mượt mà với Frontend.

5.1.2 Dịch vụ Dự đoán Thời gian Giao hàng (ETA Estimation)

Mô hình ETA (Estimated Time of Arrival) dự đoán chính xác thời gian hoàn thành chuyến bay giao hàng bằng drone dựa trên các yếu tố môi trường và vận tải.

Mô hình dự toán: Sử dụng thuật toán *XGBoost Regression* kết hợp với mạng *Multilayer Perceptron (MLP)*.

Dữ liệu đầu vào (Features): Khoảng cách tọa độ GPS (Haversine distance), khối lượng hàng hóa (kg), dung lượng pin ban đầu (%), tốc độ gió (m/s), và điều kiện thời tiết (mưa, mù).

Kết quả: Mô hình đạt độ chính xác cao với chỉ số Sai số tuyệt đối trung bình $MAE < 1.8$ phút trên tập dữ liệu kiểm thử.

5.1.3 Dịch vụ Tóm tắt Báo cáo Vận hành (Summarization)

Dịch vụ này tự động tổng hợp các nhật ký chuyến bay (flight logs), sự cố kỹ thuật hoặc báo cáo giao hàng hằng ngày cho người quản trị (Admin/Operator).

Kỹ thuật áp dụng: Mô hình Transformer tự chế bản nhẹ (Fine-tuned BART/PEGASUS) tối ưu hóa cho bài toán tóm tắt văn bản ngắn.

Đầu ra: Tóm tắt danh sách 100+ chuyến bay thành báo cáo điểm tin dài 3-5 câu: tổng số đơn thành công, số sự cố sương mù/gió lớn, và drone cần bảo trì.

5.2 Đóng Gói Hệ Thống Và Hạ Tầng DevOps

Để đảm bảo tính nhất quán giữa môi trường phát triển (Development) và môi trường triển khai thực tế (Production), toàn bộ hệ thống được đóng gói bằng công nghệ Docker.

5.2.1 Cấu trúc Đóng gói Docker Container

Hệ thống được chia nhỏ thành các Microservices đóng gói độc lập:

AI-Services Container: Chạy ứng dụng FastAPI chứa mô hình ETA, Chatbot và Summarization.

Backend Container: Xử lý logic nghiệp vụ và quản lý đội bay Drone.

Database Container: Cơ sở dữ liệu PostgreSQL / MongoDB lưu trữ hành trình.

5.2.2 Định hình tệp Dockerfile và Docker Compose

Dưới đây là cấu trúc tệp `docker-compose.yml` dùng để khởi chạy đồng bộ toàn bộ hạ tầng dịch vụ:

```
version: '3.8'

services:
  ai_service:
    build:
      context: ./ai_service
      dockerfile: Dockerfile
    container_name: drone_ai_service
    ports:
      - "8000:8000"
    environment:
      - MODEL_PATH=/app/models/eta_xgb.pkl
    restart: always

  web_backend:
    build: ./backend
    container_name: drone_backend
    ports:
      - "5000:5000"
    depends_on:
      - ai_service
      - db

  db:
    image: postgres:15-alpine
    container_name: drone_db
    environment:
      POSTGRES_USER: drone_admin
      POSTGRES_PASSWORD: secure_password
      POSTGRES_DB: drone_delivery_db
    ports:
      - "5432:5432"
```

5.3 Kiểm Thử Phần Mềm (Software Testing)

Công tác QA (Quality Assurance) được thực hiện toàn diện thông qua các cấp độ kiểm thử nhằm đảm bảo tính an toàn tối đa cho hệ thống giao hàng tự động.

5.3.1 Kế hoạch và Phương pháp Kiểm thử

Unit Testing (Kiểm thử đơn vị): Sử dụng framework `pytest` để kiểm thử các hàm tính khoảng cách, thuật toán dự báo ETA và các hàm xử lý chuỗi của Chatbot.

Integration Testing (Kiểm thử tích hợp): Kiểm tra khả năng tương tác API giữa Backend và các mô hình AI Service.

Performance & Load Testing (Kiểm thử hiệu năng): Sử dụng công cụ `Locust` để giả lập 500 yêu cầu đồng thời truy vấn ETA và Chatbot.

5.3.2 Bảng Kết quả Kiểm thử Tổng hợp

Bảng 5.1: Bảng tổng hợp kết quả kiểm thử các mô-đun chính

STT	Mô-đun kiểm thử	Kịch bản kiểm thử (Test Case)	Trạng thái
1	AI - ETA Service	Dự đoán thời gian với dữ liệu GPS chuẩn	PASS
2	AI - ETA Service	Xử lý ngoại lệ khi thiếu tham số tốc độ gió	PASS
3	AI - Chatbot	Trả lời câu hỏi tra cứu đơn hàng hợp lệ	PASS
4	API Gateway	Tải đồng thời 200 request/giây vào AI Microservice	PASS
5	Docker Environment	Khởi chạy thành công toàn bộ container qua Compose	PASS

5.4 Kết Luận Và Hướng Phát Triển

5.4.1 Kết luận

Chương này đã hoàn thành toàn bộ các mục tiêu đề ra cho vai trò AI & QA/DevOps:

Triển khai thành công 3 dịch vụ AI cốt lõi (Chatbot RAG, Dự đoán ETA bằng XGBoost/MLP, Tóm tắt báo cáo nhật ký bay).

Chuẩn hóa hạ tầng triển khai thông qua đóng gói Docker và Docker Compose, giúp hệ thống dễ dàng mở rộng.

Đảm bảo độ tin cậy và chất lượng phần mềm thông qua quy trình kiểm thử đơn vị, tích hợp và kiểm thử hiệu năng nghiêm ngặt.

5.4.2 Hướng phát triển

Trong các giai đoạn tiếp theo, hệ thống có thể được cải tiến theo các hướng:

Tích hợp hạ tầng CI/CD tự động (GitHub Actions / GitLab CI) để tự động hóa khâu kiểm thử và deploy.

Nâng cấp mô hình ETA bằng cách tích hợp thêm dữ liệu luồng không lưu thời gian thực (Real-time Air Traffic Data).

Triển khai cụm Kubernetes (K8s) để tự động cân bằng tải và tự khôi phục (auto-healing) khi số lượng Drone tăng lên quy mô lớn.